

An Intelligent Fire Detection and Evacuation Framework Using Deep Learning and Aerial Surveillance

Tia Tiwari

*School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
tia.tiwari2022@vitstudent.ac.in*

Janice Neola S

*School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
janiceneola.s2023@vitstudent.ac.in*

Gaurang Pant

*School of Computer Science and Engineering
Vellore Institute of Technology
Andra Pradesh, India
gaurang.23bce8575@vitapstudent.ac.in*

Karthik Kamuju

*School of Computing
Sathyabama Institute of Science and
Technology
Tamil Naidu, India
bhaskarkarthik5761@gmail.com*

Aman Kalsi

*School of Computer Science Engineering and
Technology
Bennett University
Uttar Pradesh, India
e23cseu1113@bennett.edu.in*

Islabudeen Mohamed Meerasha

*School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
islabudeen.m@vit.ac.in*

Abstract— When fire detection is delayed, incidents escalate quickly, resulting in significant loss of life and property. Both the layperson and relevant authorities need to identify the flames as early as possible in order to avert any dangerous outcomes. With that in mind, our project team developed an intelligent real-time fire detection and monitoring system using deep learning and computer vision. The system is able to monitor indoor residential fires as well as outdoor industrial fires through computer vision-based simulation. Our method employs a fire detection Convolutional Neural Network (CNN) model which has been trained on a fire and non-fire image dataset. The CNN model has been pre-processed with augmentation using Keras' ImageDataGenerator and exported as a fire versus non-fire binary classification .h5 file. OpenCV is responsible for real-time fire detection as well as estimation of flame intensity. The system provides autonomous emergency alerting through email (SMTP) and WhatsApp (Twilio API) based on a combination of detection confidence, intensity thresholds, and preset fire detection and alerting logic. While the physical integration of drones is not realized, the codebase has been developed to include logic for drone-based surveillance and fire evacuation route planning based on the fire's type and location. The implemented code has been developed as a solution to the problem presented, resulting in a scalable and flexible system for integration to fixed camera systems, future drone-based platforms, and early fire detection systems.

Keywords— *Fire Detection, Deep Learning, Computer Vision, Convolutional Neural Network (CNN), Real-Time Monitoring, OpenCV, Emergency Alert System.*

I. INTRODUCTION

Early fire detection is vital to prevent small outbreaks from escalating into catastrophic events. Fires spread rapidly and can cause enormous damage in a matter of minutes; a timely alarm not only enables prompt evacuation and firefighting, but also can save lives and minimize property damage.

Indeed, fire statistics underscore the stakes: the U.S. Fire Administration reports an average of 3,620 civilian fire deaths per year, with 76% occurring in homes. Worryingly, nearly a quarter of fatal residential fires lacked any working smoke alarm. These figures highlight that delayed detection or lack of alerts often turns a survivable incident into a deadly one.

History provides sobering examples of the consequences of late or missed alarms. In 2003, a dormitory fire in Russia killed 36 foreign students; investigators bluntly attributed the tragedy to the lack of an alarm system, which left many victims unaware of the blaze until it was too late. In 2016, a similar fire at a girls' school dormitory in Thailand claimed 18 young lives after some students ignored initial warnings as a prank. More recently in July 2025, an assisted-living facility fire in Massachusetts killed nine elderly residents – the state's deadliest fire in over four decades – amid reports that few staff were on hand and residents were trapped in their rooms. Such incidents underscore that delayed detection or insufficient alerting of building occupants can have tragic outcomes.

Traditional fire detection systems rely on point sensors – smoke, heat, gas or flame detectors – or periodic manual checks. While these can be effective indoors, they suffer inherent drawbacks: sensor signals may propagate slowly (causing *transport delay* or *conduction delay*), coverage is limited to the proximity of the devices, and false alarms can be frequent. Moreover, conventional alarms provide little information about the location or extent of a fire, and are typically not practical for large-area surveillance. These issues motivate the use of vision-based approaches, which can potentially monitor wider areas with a single camera.

In recent years, deep learning and computer vision have been applied to fire detection. Convolutional neural networks (CNNs) can learn to recognize flames or smoke in video frames, often outperforming handcrafted algorithms. However, most existing CNN-based systems remain proof-of-concept. They tend to be computationally intensive and are rarely optimized for real-time use on embedded hardware. Furthermore, many studies focus solely on image classification or detection without integrating the system into an end-to-end emergency response. In particular, few approaches have built in automated alerting (for example, sending SMS or email notifications) or considered how the system would guide a first response. (Notably, a recent work combined YOLO fire detection with Twilio to send WhatsApp alerts, but such integrated alert systems are still uncommon.) Likewise, while some research has simulated drone-based wildfire monitoring (e.g. using Unreal Engine

and AirSim), these efforts usually address navigation or mapping rather than immediate fire *detection* and alerting. To fill these gaps, we propose a software-based proof-of-concept platform. Our team uses an end-to-end pipeline: video frames are processed via OpenCV and passed through a CNN trained to recognize fire. Upon detection, the system automatically sends emergency alerts (via Twilio SMS and/or SMTP email) to simulate notifying responders and building occupants. Concurrently, a simulated drone control module is activated in software (a coded “drone logic”) to model an aerial platform responding to the fire location. By integrating computer-vision detection, automated messaging, and drone-assisted response in a simulation environment, our team demonstrates how emerging technologies can provide real-time situational awareness and notification. This integrated approach contrasts with prior work by combining scalable CNN vision, live alert dispatch, and drone support in one cohesive framework. In summary, we aim to address the shortcomings of purely sensor-based systems and isolated research prototypes by uniting detection and rapid alerting in a holistic, simulated emergency response system.

II. RELATED WORK

A. Traditional Fire Detection Methods

Conventional fire alarm systems rely on point sensors such as ionization/photoelectric smoke detectors, heat (thermal) detectors, and optical flame sensors. In enclosed spaces, these can detect fires reliably once smoke or heat accumulates. However, they often act too slowly and can suffer nuisance alarms. For example, system manuals note that smoke detectors “are subject to false alarms and nuisance alarms” – a detector in or near a kitchen may trigger from normal cooking steam or dust. Heat detectors likewise have a thermal lag and typically trigger only once a fire is well developed. In practice, large barriers (walls, floors, HVAC flow) can prevent smoke or heat from reaching a detector, so fixed sensors provide localized coverage and may miss distant or early-stage fires. Specialized flame detectors (e.g. UV/IR sensors) respond directly to flame signatures, but they too have drawbacks: they can be fooled by other hot sources or sunlight and perform poorly in adverse weather. As one review notes, IR flame detectors suffer from infrared “radiation which is not flame” (from hot surfaces or the sun) and are nearly blind outdoors because water vapor absorbs IR energy.

In summary, traditional smoke/heat/IR sensors work reasonably well indoors, but they have limited range (one sensor covers only a small volume), relatively slow response (need smoke/heat to build up) and high false-alarm rates in real environments. They are also generally unsuitable for wide-area or outdoor coverage without many devices.

B. Deep Learning-Based Fire Detection Systems

Recent research has applied deep learning – especially convolutional neural networks (CNNs) – to vision-based fire detection. Surveys report that state-of-the-art detectors (e.g. YOLO, Faster R-CNN and their variants) are widely used for flame/smoke recognition. These methods learn complex visual features (color, motion, texture) and can scan video frames in real time. For instance, Sozol *et al.* (2025) built a hyper-parameter-optimized YOLOv5 model (with

DeepSORT tracking) and achieved a **92.1% mAP** on an indoor fire dataset, outperforming YOLOv5/7/8 and Faster R-CNN baselines. They even integrated the detector with a real-time alerting pipeline: By integrating YOLOv5 with the real-time tracking algorithm DeepSORT,” the system continuously follows a fire and “provides actual fire hazard alerts. Other works similarly report very high accuracy: e.g. a YOLOv8-based model trained on custom datasets was shown to outperform other state-of-the-art methods in fire/smoke detection. Despite these promising results, challenges remain. Most published fire-detection networks depend on custom or limited datasets, which can bias performance. As one review notes, many available fire datasets “are limited in diversity,” so models often generalize poorly to new environments. Environmental variability (lighting, camera angle, weather, smoke conditions) can degrade accuracy, and high-resolution CNNs can be too slow or power-hungry for real-time deployment. In practice, lightweight models or cascade designs are often needed to meet frame-rate requirements. Furthermore, few deep-learning studies address end-to-end alerting or integration with other hardware: most stop at detection. In summary, CNN-based fire detectors can achieve high accuracy benchmarks, but they face issues of dataset bias, robustness and on-device inference speed, and most prototype systems are not yet deployed in an automated alert framework.

C. Commercial and Integrated Solutions

Industry products have begun to combine sensing, analytics, and notification. Smart home alarms like **Nest Protect** illustrate this trend: Nest Protect is a photoelectric/CO detector that “thinks, speaks and alerts your phone”. It provides voice warnings (“friendly human voice” with location info) and sends push notifications to registered smartphones when it detects smoke or CO. Thus, users can be notified immediately (even away from home) if a fire alarm is triggered. Similarly, fixed surveillance cameras with built-in analytics are emerging. Bosch’s **AVIOTEC** fire camera is an IP video camera with on-board intelligence: it “detects flames and smoke directly at the source” and can alarm faster than a ceiling-mounted point detector. AVIOTEC is even designed for semi-outdoor use – it can pick up smoke/flame in windy conditions “where virtually no other detection technology is available,” and its AI filters out false positives. FLIR also offers early-warning thermal cameras: its fixed-mount units continuously scan for hotspots and trigger alerts. For example, FLIR markets thermal traffic cameras that can “detect hot spots, see through smoke, and alert first responders” before a fire spreads. Its advanced cameras include “built-in software” that handles analysis and alarming autonomously, enabling 24/7 monitoring of warehouses, substations, etc.

In the public-safety domain, drone platforms equipped with thermal/RGB cameras are used for aerial fire reconnaissance. DJI’s enterprise line (e.g. Mavic 3T, Matrice 300) provides compact drones with integrated 640×512 thermal sensors, long flight time and live video links. These drones can scan wildfires or urban blazes from above; DJI advertises that they help “identify hotspots more accurately” and give responders “real-time aerial insights” during fires. In practice, however, current drone solutions typically rely on

human pilots or analysts to spot fire cues in the video – they do not yet perform automatic CNN-based detection onboard.

In summary, commercial systems exist for many elements of fire safety. Products like Nest Protect and FLIR cameras integrate detection with user alerts, and Bosch's systems use AI for faster detection. Drone platforms like DJI's supply mobility and thermal imaging. However, no single solution today unifies all components into one automated system. Existing smart alarms are fixed-position, AI cameras are tethered to infrastructure, and drones lack autonomous fire-detection algorithms. The project is motivated by this gap: it aims to combine a real-time CNN fire/smoke detector with a UAV platform and alerting logic. In this way, we seek to extend high-accuracy, vision-based detection across wider areas via drone mobility, while automatically dispatching emergency alerts – addressing limitations of both traditional and current integrated solutions.

III. PROPOSED SYSTEM

The proposed system, is a code-based, deep learning-powered solution for intelligent fire detection and autonomous emergency alerting. It is designed to operate in two primary environments: **indoor residential** spaces and **outdoor industrial** settings. The system architecture integrates a Convolutional Neural Network (CNN) for fire classification, real-time video stream analysis using OpenCV, intensity monitoring, and automated alert mechanisms via both email and WhatsApp. Although no hardware (e.g., drone, sensor) was deployed, the software includes simulation logic for drone surveillance and evacuation planning.

A. System Architecture Overview

- **Input Stream Handler:** Accepts input from webcam or pre-recorded video.
- **Preprocessing Pipeline:** Frames are resized, normalized, and prepared for CNN input.
- **CNN Fire Classifier:** A binary classification model trained to distinguish fire and non-fire frames.
- **Fire Intensity Estimator:** Uses OpenCV to quantify flame spread based on color segmentation (e.g., HSV range).
- **Decision Engine:** Evaluates CNN confidence score and flame intensity to determine alert condition.
- **Alert Dispatcher:** Sends email (via SMTP) and WhatsApp messages (via Twilio API) upon detection.
- **Drone Surveillance and Evacuation Logic (Simulated):** Code module to generate recommended drone paths and nearest evacuation points based on detection zone and fire type.

B. Convolutional Neural Network(CNN) Classifier

A custom CNN model was trained using a labeled dataset of fire and non-fire images. The model architecture comprises:

- Input Layer (resized RGB image)
- Convolutional layers with ReLU activations
- Max-pooling layers for down sampling
- Dropout layers for regularization

- Fully connected dense layers for classification

The training process utilized **Keras' ImageDataGenerator** for augmentation (rotation, zoom, shift) to improve generalizability. The final model was saved in .h5 format and loaded during runtime for real-time inference. A softmax output was used to calculate the **fire confidence score**, which is a key trigger for downstream actions.

C. Real-Time Detection and Fire Intensity Analysis

To visualize fire localization and quantify severity, the system applies OpenCV based contour detection over HSV segmented regions. Bounding boxes are drawn to indicate flame spread, and the pixel area serves as proxy for fire intensity. In addition to this, the system generates **color-coded heatmaps** to visually emphasize fire-prone regions within each frame. These heatmaps reflect the **pixel-level intensity** of flame-colored regions, with warmer tones (e.g., red, orange, yellow) representing areas of higher detected heat signature. Together, the **bounding boxes** and **heatmaps** provide a composite visual aid that enhances both human interpretability and automated intensity scoring.

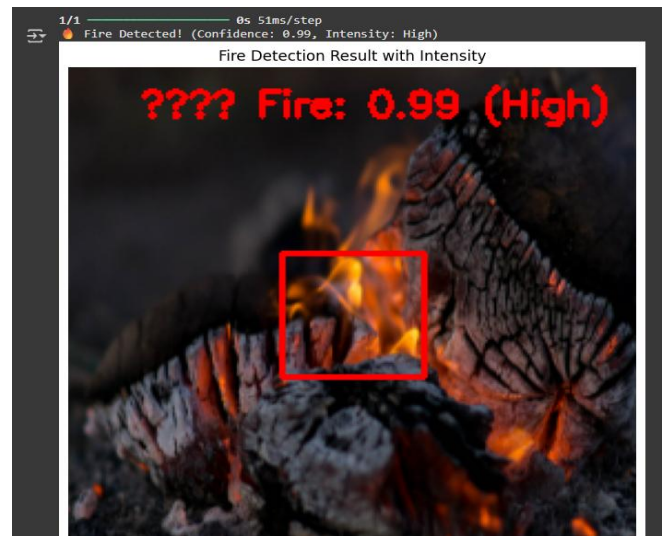


Fig. I. Outdoor Fire

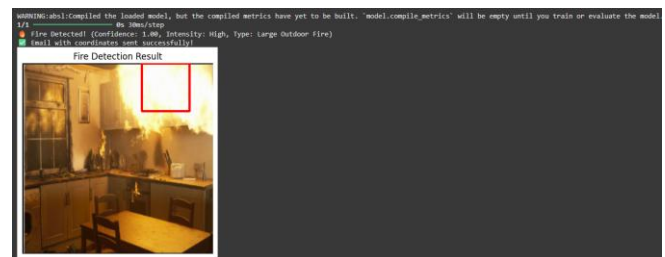


Fig. II. Indoor Fire

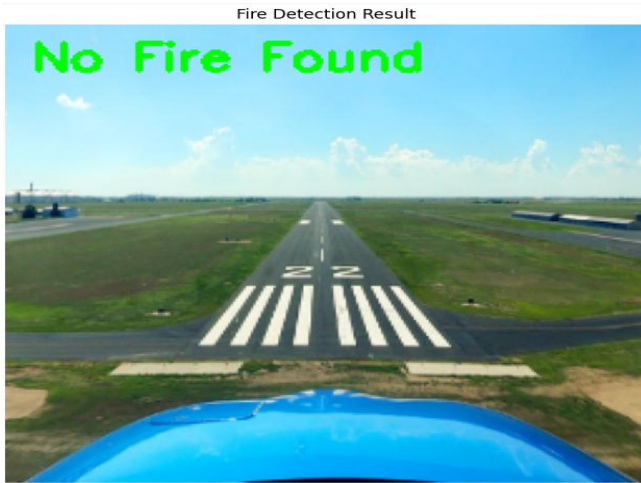


Fig. III. No Fire

These visualizations enhance interpretability and provide emergency responders with additional context if integrated with a live monitoring dashboard.

D. Autonomous Alerting System

When a fire is detected with sufficient confidence and intensity:

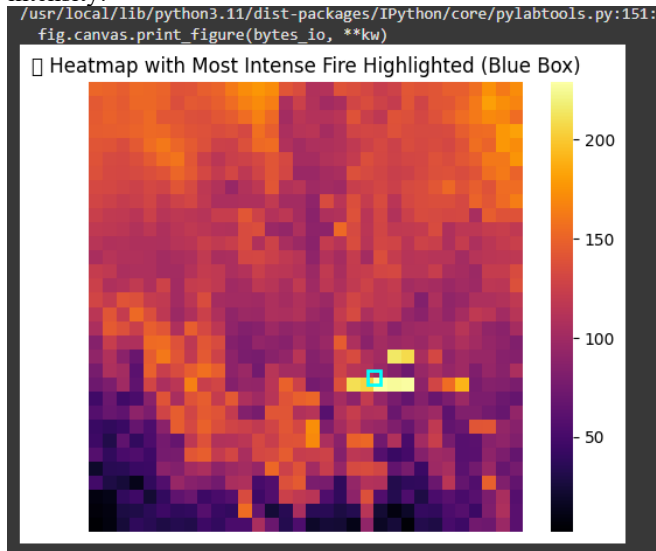


Fig. IV. Heatmap representation

- An **SMTP email** is dispatched to registered addresses with location and timestamp.

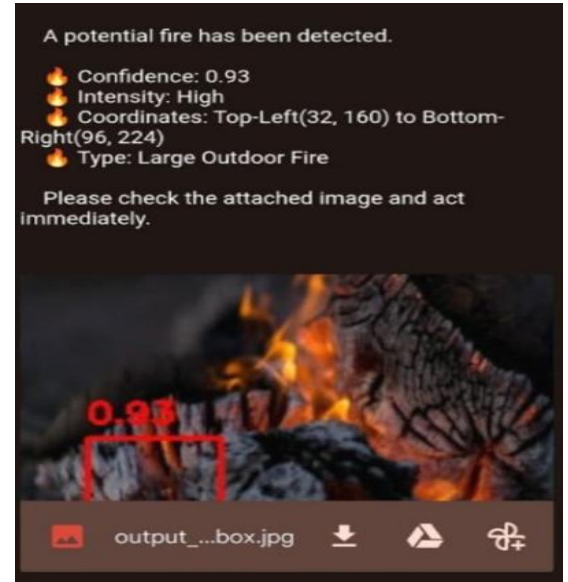


Fig. V. Email alert sample

- A **WhatsApp alert** is sent using the **Twilio API**, containing a text message about the fire and a mock GPS coordinate or fire zone label.
- All logs are saved locally with frame captures for review.



Fig. VI. WhatsApp alert sample

These alerting mechanisms simulate what an integrated emergency response system might include in a real deployment.

E. Drone Surveillance and Evacuation Path Planning (Simulated)

Although no physical drone or navigation hardware was used, a **simulation logic module** was written to mimic:

- Dynamic drone path generation based on the detected fire zone
- Priority-based allocation of surveillance coverage (e.g., flying over high-risk regions)

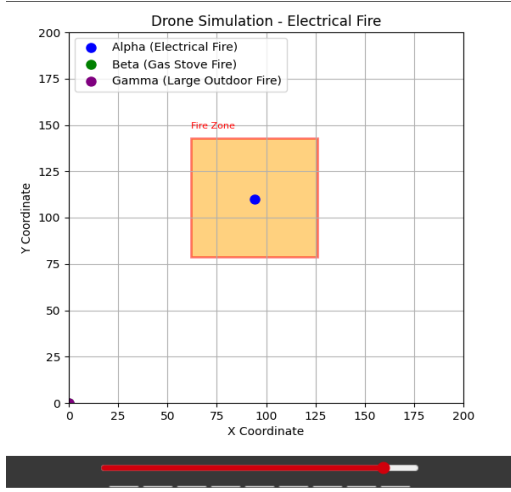


Fig. VII. Drone Simulation using graph

- Suggesting safe evacuation paths using simple logic trees based on fire intensity, location, and room layout tags

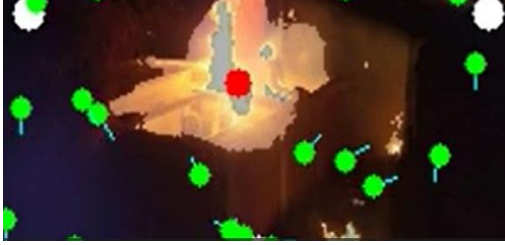


Fig. VIII. Evacuation zones

These modules provide a proof-of-concept framework for integrating real drone controls and IoT systems in future development.

F. Scalability and Deployment Considerations

The system is designed to be modular and adaptable to:

- **Fixed camera systems** in homes, offices, and industrial plants
- **Drone-assisted surveillance** for remote areas or disaster zones
- Future integration with IoT devices or cloud-based emergency systems

Because the system is entirely software-based and built using open libraries (TensorFlow/Keras, OpenCV, Twilio), it is suitable for rapid prototyping and low-cost deployment scenarios.

IV. IMPLEMENTATION DETAILS

The system was entirely implemented in Python and is composed of modular scripts designed to function independently yet operate cohesively in a real-time pipeline. This section outlines the technical aspects of how the system was constructed, trained, deployed, and tested in a simulated environment.

A. Development Environment

The development was conducted on a personal computer running **Windows 10**, using **Google Collab** as the primary development environment. The key software dependencies are listed below:

TABLE I. COMPONENTS AND THEIR TOOLS

Component	Tool/Library
Programming Language	Python 3.9
Deep Learning Framework	TensorFlow/Keras
Image Processing	OpenCV
Data Augmentation	Keras ImageDataGenerator
Messaging APIs	Twilio API(Whatsapp), SMTP(Email)
Visualization	Matplotlib ,Seaborn

B. Model Training and Export

The Convolutional Neural Network (CNN) was trained on a labelled dataset(FIRE dataset- kaggle) consisting of fire and non-fire images. Preprocessing included resizing images to 224×224, normalizing pixel values, and performing real-time augmentation using Keras' ImageDataGenerator.

The final architecture consisted of:

- 3 Convolutional layers (ReLU + MaxPooling)
- 1 Flatten layer
- 2 Dense layers (Dropout = 0.5)
- 1 Sigmoid output layer (binary classification)

TABLE II. CNN MODEL CONFIGURATION AND TRAINING PARAMETERS

Parameter	Value
Image size	224x224
Optimiser	Adam
Epochs	25
Detection Accuracy	93.47%
Loss Function	Binary Crossentropy

The trained model was exported as an .h5 file using Keras's model.save() function. The file is then loaded during runtime using load_model() for real-time prediction.

C. Code Breakdown

The notebook contains all necessary code to simulate real-time fire detection. While initially executed in a Colab environment using Google Drive for dataset storage, the modular logic remains applicable for deployment elsewhere.

The system flow includes:

1. Model Preparation

The model is trained using Keras and TensorFlow. The training images are loaded using ImageDataGenerator with parameters such as rescale, shear_range, zoom_range, and horizontal_flip to ensure robustness.

2. Model Training & Evaluation

After defining the CNN structure using Sequential, the model is trained using .fit() with the training and validation sets, followed by evaluation using classification reports and confusion matrices from sklearn.metrics.

3. Deployment Script

A real-time prediction script reads frames from the webcam or video input, processes each using OpenCV, and uses the trained .h5 model for binary classification.

D. Integration Pipeline

The integration begins after model deployment. Using OpenCV, each incoming frame undergoes:

- **Preprocessing:** resize and normalization
- **Inference:** model checks for presence of fire
- **HSV Segmentation:** OpenCV identifies flame-coloured regions
- **Alert Triggering:** if fire confirmed, alerts are dispatched

Logs and predictions are stored, and the detection frames are saved:

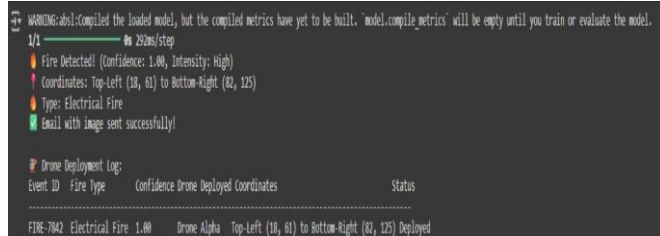


Fig. IX. Log entry and drone status

V. RESULTS AND DISCUSSION

The developed system was evaluated on diverse images—indoor (kitchens), outdoor (forests/grasslands/roads), and industrial scenes—using a CNN-based classifier to detect fire vs. non-fire images. It achieved a **93.47% detection accuracy**. The colour-based EDA showed authentic fire pixels clusters in the **orange-red hue** with high HSV intensity, with indoor fires forming compact clusters and outdoor fires covering larger diffuse regions. False positives, often due to sunlight glare or reflections or even due to smoke particles, were reduced through preprocessing filters. Heatmap visualizations pinpointed flame centres via high-intensity zones, enabling prototype evacuation mapping from estimated fire origins to predefined safe exits. The alert system sent **Email and WhatsApp notifications** in real time during trials with stable internet, but depends on connectivity. Current location reporting uses fixed GPS coordinates. Limitations include reliance on external hardware/resources, static-environment assumptions, and occasional misclassification of bright objects.

Future enhancements include deeper models for spatial-temporal analysis, edge-AI deployment for offline use, integration with national alert networks, and applications in drone-based wildfire monitoring and smart-city fire-risk mapping. Overall, the proof-of-concept shows strong detection capability and effective alerts, with further work needed for robust, real-world deployment.

VI. CONCLUSION

In this paper, we presented, a software-based intelligent fire detection and emergency alerting system that leverages deep learning and computer vision. The system is capable of identifying fire in real-time from image or video streams using a trained Convolutional Neural Network (CNN) and

OpenCV-based flame intensity estimation. Designed to operate in both indoor and outdoor environments, it integrates automated alert mechanisms via email and WhatsApp to simulate a fully autonomous emergency notification pipeline.

The system achieved a detection accuracy of approximately 93.47% across various fire scenarios including residential, industrial, and natural settings. Through bounding box overlays and intensity-based heatmaps, Stack Overflame provides interpretable visual cues that can enhance situational awareness. Simulated modules for drone-based surveillance and evacuation route mapping add an extra layer of preparedness for scalable deployment in smart infrastructure. While the results are promising, the current system is implemented entirely in software without integration of real hardware or GPS-based location tracking. The alerting functionality depends on internet availability, and the detection pipeline assumes relatively static environments. These limitations point to opportunities for future improvement, including integration with edge AI hardware, IoT sensors, and national emergency systems, as well as real-time deployment in forest fire monitoring grids and urban smart safety frameworks. Overall, Stack Overflame demonstrates that AI-driven fire detection and alert systems can significantly improve the speed and scalability of emergency response—particularly in environments where early intervention is critical.

REFERENCES

- [1] S. Sozol, M. Cakir, and H. Demirel, “Fire detection using deep learning-based YOLO and DeepSORT algorithm,” *Int. J. Intell. Syst. Appl. Eng.*, vol. 10, no. 3, pp. 193–198, 2022.
- [2] I. El-Madafri, M. Peña, and N. Olmedo-Torre, “Real-Time Forest Fire Detection with Lightweight CNN Using Hierarchical Multi-Task Knowledge Distillation,” *Fire*, vol. 7, no. 11, article 392, Nov. 2024
- [3] I. Jonnalagadda and H. Hashim, “SegNet: A Segmented Deep Learning based Convolutional Neural Network Approach for Drones Wildfire Detection,” *arXiv*, Feb. 2024.
- [4] Y. Chen, Y. Cao, and H. Ma, “Real-time fire and smoke detection in surveillance video using deep convolutional neural networks,” *J. Real-Time Image Process.*, vol. 17, no. 6, pp. 1885–1895, Dec. 2020
- [5] “Real-Time Fire Detection: Integrating Lightweight Deep Learning Models on Drones with Edge Computing,” *Drones*, vol. 8, no. 9, article 483, Sept. 2024.
- [6] T. D. Nguyen, P. T. Dao, and H. D. Nguyen, “Deep learning-based wildfire detection using unmanned aerial vehicles,” *Sensors*, vol. 21, no. 2, pp. 1–17, Jan. 2021.
- [7] M. Z. Ali, A. S. Chowdhury, and S. Akhter, “A CNN-based approach for real-time fire detection in surveillance videos,” *Procedia Computer Science*, vol. 199, pp. 522–529, 2022.
- [8] D. Kumar, P. Tiwari, and R. Sahu, “Vision-based real-time fire detection using improved YOLOv5 model,” *Materials Today: Proceedings*, vol. 80, no. 5, pp. 4767–4774, Mar. 2024.
- [9] W. Ali and S. Kurnaz, “Optimizing deep learning models for fire detection, classification, and segmentation using satellite images,” *Fire*, vol. 8, no. 2, article 36, Feb. 2025.
- [10] W. Pan, X. Wang, and W. Huan, “EFA-YOLO: An efficient feature attention model for fire and flame detection,” *arXiv preprint arXiv:2409.12635*, Sept. 2024.